

TP1

Structures de données, qualité de code et tests

Structures de données et qualité de code

Contraintes et Remise du TP

- La correction tiendra compte de la brièveté des fonctions que vous écrivez (évités les fonctions de plus de 25 lignes) - n'hésitez pas à découper une fonction en plusieurs sous-fonctions plus courtes.
- Vos fonctions doivent être documentées (docstring).
- Pensez à utiliser pylint pour avoir un code qui soit le plus qualitatif possible.
- Les noms de variables et de fonctions doivent être explicites et en minuscules (snake_case).
- Ce TP est un travail **personnel** (pas de binôme) : chacun doit produire et rendre son propre code.
- Le rendu se fait exclusivement via votre **dépôt GitHub** personnel. Vous avez 4h pour faire le TP, et le commit pris en compte sera le dernier réalisé avant minuit.

Exercice 1: Rendu via GitHub (5 points)

Ce TP étant un travail personnel, vous devez le rendre via votre dépôt GitHub TP_OOP, déjà créé lors d'un TP précédent. Cette question met en place ce rendu avant de commencer à coder.

1. **1 point** Récupérez la dernière version de votre dépôt TP_OOP, et créez-y un nouveau dossier `tp0/` qui contiendra exclusivement les fichiers de ce TP (un dossier par TP permet de garder un dépôt propre et lisible).

```
git clone https://github.com/<votre-identifiant>/TP_OOP.git
cd TP_OOP
git pull
mkdir tp0
cd tp0
```

2. **1 point** Vérifiez que la racine du dépôt contient bien un fichier `.gitignore` adapté à un projet Python; si ce n'est pas déjà le cas depuis un TP précédent, ajoutez-le (au minimum `__pycache__/, *.pyc`, et le dossier de votre environnement virtuel).
3. **2 points** Dans le dossier `tp0/`, organisez votre code en un fichier par question (`tuples.py`, `ensembles.py`, `dictionnaires.py`, `qualite.py`, `tests.py`), et effectuez un commit **après chaque question terminée**, avec un message explicite décrivant ce qui a été fait.

```
git add tp0/tuples.py
git commit -m "Question 3 : tuples pour le journal de bord du robot"
git push
```

4. **1 point** Avant la remise, vérifiez que l'historique de votre dépôt reflète bien votre progression au fil du TP, et non un unique commit final.

```
git log --oneline
```

Assurez-vous que votre dernier commit est bien poussé sur GitHub (`git push`) avant minuit : c'est ce commit qui sera corrigé.

Exercice 2: Environnement virtuel avec Conda (4 points)

Avant de commencer à coder, vous allez isoler les dépendances de ce TP dans un environnement virtuel dédié, comme vu en cours.

1. **1 point** Si vous n'avez pas encore d'environnement dédié à ce module, créez un environnement conda nommé `tp_oop` utilisant Python 3.11, puis activez-le. Si vous en avez déjà créé un lors d'un TP précédent, activez-le simplement.

```
conda create -n tp_oop python=3.11
conda activate tp_oop
```

2. **1 point** Installez `pylint` dans cet environnement (nécessaire pour la Question 6 sur la qualité de code).

```
conda install conda-forge::pylint
```

3. **1 point** Exportez la configuration de votre environnement dans un fichier `environment.yml` à la racine de votre dépôt TP_OOP (pas dans `tp0/`, cet environnement étant partagé entre tous les TP du module), afin qu'il soit reproductible par le correcteur.

```
conda env export > environment.yml
git add environment.yml
git commit -m "Ajout de l'environnement conda du TP"
```

4. **1 point** Avant de lancer vos tests ou `pylint`, vérifiez que vous travaillez bien dans le bon environnement.

```
conda env list
which python
```

Exercice 3: Journal de bord d'un robot avec des tuples (4 points)

Un robot explorateur enregistre à intervalles réguliers des relevés de ses capteurs lors d'une phase d'étalonnage. Chaque relevé, une fois pris, ne doit plus pouvoir être modifié accidentellement : vous allez donc les représenter par des tuples.

1. **2 points** Représentez chaque relevé par un tuple (`nom_capteur`, `valeur`, `unite`). Constituez une liste `releves` contenant au moins trois relevés, et implémentez une fonction `afficher_releve` qui retourne une chaîne de caractères décrivant un relevé.

```
releve1 = ("laser_avant", 2.35, "m")
releve2 = ("laser_arriere", 1.10, "m")
releve3 = ("gyroscope", 87.5, "deg")
releves = [releve1, releve2, releve3]

assert len(releves) == 3
assert releves[0][0] == "laser_avant"
assert afficher_releve(releve1) == "Capteur laser_avant : 2.35 m"
```

2. **2 points** Après un nouvel étalonnage, la valeur d'un capteur doit être mise à jour. Les tuples étant immuables, implémentez `recalibrer` qui reconstruit la liste des relevés avec la nouvelle valeur pour le capteur concerné, et laisse les autres relevés inchangés.

```
nouveaux_releves = recalibrer(releves, "laser_avant", 2.40)

assert nouveaux_releves[0] == ("laser_avant", 2.40, "m")
assert nouveaux_releves[1] == releve2
assert nouveaux_releves[2] == releve3
```

Exercice 4: Coordination d'une flotte de robots avec des ensembles (4 points)

Un entrepôt coordonne plusieurs robots affectés à différentes missions. Certains robots peuvent être affectés à plusieurs missions à la fois ; le responsable souhaite pouvoir facilement identifier qui fait quoi, sans doublons.

1. **2 points** Créez deux ensembles `robots_exploration` et `robots_transport`, puis implémentez les fonctions `robots_double_mission`, `robots_toutes_missions` et `robots_exploration_seulement`.

```
robots_exploration = {"R2", "R5", "R7"}
robots_transport = {"R5", "R9", "R7", "R3"}

double_mission = robots_double_mission(
    robots_exploration, robots_transport)
toutes_missions = robots_toutes_missions(
    robots_exploration, robots_transport)
exploration_seule = robots_exploration_seulement(
```

```

    robots_exploration, robots_transport)

assert double_mission == {"R5", "R7"}
assert toutes_missions == {"R2", "R3", "R5", "R7", "R9"}
assert exploration_seule == {"R2"}

```

2. 2 points Un robot supplémentaire rejoint l'exploration, et un autre est retiré du transport (panne). Implémentez `ajouter_robot_mission` et `retirer_robot_mission`, qui renvoient un **nouvel** ensemble sans modifier celui passé en argument.

```

ajout = ajouter_robot_mission(robots_exploration, "R8")
retrait = retirer_robot_mission(robots_transport, "R9")

assert ajout == {"R2", "R5", "R7", "R8"}
assert retrait == {"R3", "R5", "R7"}

# L'ensemble d'origine ne doit pas avoir été modifié
assert robots_transport == {"R5", "R9", "R7", "R3"}

```

Exercice 5: Inventaire de pièces détachées avec des dictionnaires (5 points)

L'atelier de maintenance gère un stock de pièces détachées pour plusieurs modèles de robots. Pour chaque modèle, on suit le nombre de moteurs, de capteurs et de roues disponibles.

1. 2 points Créez un dictionnaire `pieces_stock` où chaque clé est un nom de modèle, et chaque valeur un sous-dictionnaire des quantités disponibles. Implémentez `quantite_piece` qui retourne la quantité disponible d'une pièce pour un modèle donné.

```

pieces_stock = {
    "ModeleA": {"moteurs": 10, "capteurs": 25, "roues": 40},
    "ModeleB": {"moteurs": 6, "capteurs": 15, "roues": 24},
}

assert quantite_piece(pieces_stock, "ModeleA", "moteurs") == 10

```

2. 3 points Implémentez `consommer_piece` (retire des pièces du stock après une réparation), `ajouter_modele` (enregistre un nouveau modèle de robot avec son stock initial), et `total_pieces` (calcule, tous modèles confondus, le nombre total de moteurs, de capteurs et de roues).

```

consommer_piece(pieces_stock, "ModeleA", "moteurs", 3)
assert pieces_stock["ModeleA"]["moteurs"] == 7

ajouter_modele(pieces_stock, "ModeleC",
               moteurs=4, capteurs=10, roues=16)
assert pieces_stock["ModeleC"] == \
    {"moteurs": 4, "capteurs": 10, "roues": 16}

totaux = total_pieces(pieces_stock)
assert totaux == {"moteurs": 17, "capteurs": 50, "roues": 80}

```

Exercice 6: Qualité de code (2 points)

Un binôme pressé a écrit la fonction ci-dessous, reprise du TP précédent, pour calculer le coût énergétique d'un déplacement selon le type de terrain.

```

def f(d, t, x1, y1, x2, y2):
    dist = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
    if t == 'R':
        c = dist * 1.0
    elif t == 'H':
        c = dist * 1.5
    elif t == 'S':
        c = dist * 2.0
    else:
        c = dist * 3.0

```

```
print("cout:", c)
return c
```

1. 2 points Identifiez au moins **trois** problèmes de qualité que pylint signalerait sur cette fonction (noms non explicites, paramètre inutilisé, absence de documentation, mélange calcul/affichage, valeurs "magiques", etc.), puis proposez une nouvelle version `cout_deplacement_propre` qui corrige ces problèmes : nom et paramètres explicites, docstring, une seule responsabilité (pas d’affichage dans la fonction de calcul), et sans dépasser 25 lignes.

Exercice 7: Tests unitaires (5 points)

Vous allez maintenant tester une partie des fonctions écrites dans ce TP à l’aide du module `unittest`, en veillant à couvrir à la fois un cas usuel et un cas limite par fonction.

```
import unittest

class TestJournalDeBord(unittest.TestCase):
    """Tests pour les fonctions sur les relevés (tuples)."""

    def test_recalibrer_capteur_existant(self):
        ...

    def test_recalibrer_capteur_absent(self):
        """Cas limite : le capteur demande n'existe pas."""
        ...

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

1. 3 points Écrivez la classe `TestJournalDeBord` ci-dessus dans son intégralité, ainsi qu’une classe `TestFlotteRobots` testant `robots_double_mission` et `ajouter_robot_mission`. Pour cette dernière fonction, le cas limite attendu est l’ajout d’un robot déjà présent dans l’ensemble (le résultat ne doit pas contenir de doublon).
2. 2 points Écrivez une classe `TestInventaire` testant `consommer_piece` et `total_pieces`. Le cas limite attendu pour `total_pieces` est un stock ne contenant aucun modèle : la fonction doit alors renvoyer `{"moteurs": 0, "capteurs": 0, "roues": 0}`.

Exercice 8: Regard critique avec un assistant IA (7 points)

Les questions précédentes doivent rester votre propre travail. Celle-ci vous propose d’utiliser un assistant IA (GitHub Copilot, version gratuite) uniquement sur des tâches annexes – relecture, tests, documentation, commits – jamais pour générer une solution.

1. 1 point Activez GitHub Copilot dans votre éditeur.
2. 2 points Faites relire l’une de vos fonctions (ex. `recalibrer`), sans demander de réécriture. Notez deux remarques pertinentes, et une – s’il y en a une – incorrecte ou hors sujet.
3. 1 point Demandez un test unitaire supplémentaire pour cette fonction. Passe-t-il ? Couvre-t-il un cas que vous n’aviez pas prévu ?
4. 1 point Faites rédiger ou compléter le `README.md` du dépôt (contenu de `tp0/`, comment lancer le code et les tests), puis corrigez-le avant de le valider.
5. 1 point Utilisez la génération de message de commit de Copilot (icône dans *Source Control*) sur un commit réel. Est-il aussi précis qu’un message que vous auriez écrit vous-même ?
6. 1 point En 2-3 phrases, quel usage vous a semblé le plus fiable ? (Notée sur la réflexion, pas sur la conclusion.)